

# Tarea 2 – Física Computacional FIS205

Departamento de Física  
Universidad Técnica Federico Santa María

27 de junio de 2026

---

## 1. Problema 1: Inteligencia artificial para problemas inversos

### 1.1. a) Explicación de conceptos

**Problema directo e inverso:** En el problema conocemos  $(\gamma, k)$  y resolvemos la ecuación diferencial para obtener  $x(t)$ . Este calculo esta bien definido, a cada par de parámetros le corresponde una señal. Ahora el problema inverso es el caso contrario, a partir de la señal observada queremos  $(\gamma, k)$ . Aquí dificultad es que no existe una fórmula explícita que invierta el operador que va de los parámetros a la señal, y la señal medida siempre viene con ruido.

Dos pares de parámetros distintos pueden producir señales parecidas, sobre todo cuando el ruido domina las diferencias finas, de modo que el problema inverso suele ser mal condicionado, ya que con pequeñas perturbaciones en los datos se traduce en grandes variaciones en la estimación de los parámetros.

**Aprendizaje supervisado:** Trabajamos con un conjunto  $\{(\mathbf{x}_i, \theta_i)\}_{i=1}^N$ , donde cada señal  $\mathbf{x}_i$  está acompañada de su etiqueta  $\theta_i = (\gamma_i, k_i)$ . Si necesitamos conocer las etiquetas, ya que el modelo aprende a invertir el problema porque le mostramos ejemplos de entrada-salida y ajusta sus parámetros internos minimizando el error entre la predicción y la etiqueta real. Como en la simulación nosotros mismos generamos las señales con parámetros conocidos, nos permite plantear el problema inverso como una regresión supervisada.

**Conjuntos de entrenamiento y validación:** Separamos los datos en **train** y **test** para poder estimar cómo se comporta el modelo frente a datos que nunca vio. Ahora, si evaluáramos el desempeño sobre los mismos datos usados para entrenar, el error sería artificialmente bajo y no nos diría nada sobre la capacidad de generalización. El conjunto de validación actúa como un examen independiente, este detecta si el modelo aprendió la relación física real o si simplemente memorizó el ruido del entrenamiento.

**Función de pérdida:** Necesitamos una medida cuantitativa del error de predicción para poder optimizar el modelo. El error cuadrático medio esta dado por:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \left[ (\gamma_i - \hat{\gamma}_i)^2 + (k_i - \hat{k}_i)^2 \right] \quad (1)$$

Este penaliza las desviaciones grandes y es diferenciable, lo que lo hace conveniente para los algoritmos de optimización. Minimizar el MSE equivale a acercar las predicciones  $(\hat{\gamma}, \hat{k})$  a los

valores reales en promedio, y su raíz tiene la ventaja de quedar expresada en las mismas unidades que los parámetros.

## 1.2. b) Generación de los datos

El script construye el conjunto de datos resolviendo numéricamente la ecuación para  $N = 3000$  pares  $(\gamma, k)$  elegidos al azar. Los parámetros se muestrean desde distribuciones uniformes en los rangos:

$$\gamma \sim \mathcal{U}(0,05, 1,0), \quad k \sim \mathcal{U}(1,0, 5,0),$$

con la semilla fijada (`np.random.seed(42)`) para que el experimento sea reproducible.

Para integrar la ecuación de 2do orden lo reescribimos como un sistema de 1er orden, que es lo que requiere `odeint`. Definimos  $v = \dot{x}$ ,

$$\frac{dx}{dt} = v, \quad \frac{dv}{dt} = -\gamma v - kx, \quad (2)$$

esto en el código es la función `oscilador`. Cada EDO se resuelve sobre  $N_t = 1000$  puntos en  $t \in [0, 10]$  y se guarda solo la posición  $x(t)$ , que es la información que tendría un experimento. La velocidad se descarta porque la señal observable es la posición.

Una vez generadas las señales limpias, les agregamos un ruido gaussiano independiente punto a punto, el cual está definido por:

$$x_{\text{obs}}(t_j) = x(t_j) + \eta_j, \quad \eta_j \sim \mathcal{N}(0, \sigma^2), \quad \sigma = 0,02, \quad (3)$$

Con esto simulamos el carácter imperfecto de una medición real.

Finalmente, graficamos el efecto de cada parámetro por separado. Fijando  $k$  y variando  $\gamma$  se observa cómo cambia la rapidez del decaimiento de la envolvente, a mayor  $\gamma$ , la amplitud se amortigua más rápido. Fijando  $\gamma$  y variando  $k$  se aprecia el cambio en la frecuencia de oscilación, ya que la frecuencia natural crece con  $\sqrt{k}$ .

## 1.3. c) Entrenamiento de los modelos:

Los datos se dividen en 80 % entrenamiento y 20 % validación con `train_test_split`. Entrenamos 2 modelos de regresión que reciben las 1000 muestras de la señal como entrada y predicen los dos parámetros como salida:

- **Random Forest:** es un ensamble de árboles de decisión, cada uno entrenado sobre una muestra distinta de los datos. La predicción final es el promedio de todos los árboles, lo que reduce la varianza y lo hace robusto frente al ruido. La opción `n_jobs=-1` aprovecha todos los núcleos del procesador para entrenar en paralelo.
- **Red neuronal MLP:** aprende una función no lineal que mapea la señal completa a los parámetros físicos, ajustando sus pesos durante un máximo de 500 épocas.

El desempeño se mide con la raíz del error cuadrático medio, calculada por separado para cada parámetro mediante:

$$\text{RMSE}_\gamma = \sqrt{\frac{1}{N_{\text{test}}} \sum_i (\gamma_i - \hat{\gamma}_i)^2}, \quad \text{RMSE}_k = \sqrt{\frac{1}{N_{\text{test}}} \sum_i (k_i - \hat{k}_i)^2}. \quad (4)$$

La función `calcular_rmse` se encarga de pedir las predicciones del modelo sobre el conjunto de test y aplicar esta fórmula a cada columna.

Al ejecutar el script se observa que el error en  $k$  tiende a ser menor que el error en  $\gamma$ .

#### 1.4. d) Efecto del ruido:

Para cuantificar la sensibilidad al ruido, el script repite el entrenamiento variando el nivel de ruido con:

$$\sigma = 0, 0,01, 0,02, 0,05, 0,10,$$

Para cada valor genera una versión nueva de las señales sumando ruido a las señales limpias. Para acelerar el barrido se usa un Random Forest con 50 árboles en lugar de 100. El caso  $\sigma = 0$  se trata aparte usando directamente las señales limpias, lo que evita llamadas innecesarias al generador aleatorio.

El gráfico de  $\text{RMSE}_\gamma$  y  $\text{RMSE}_k$  en función de  $\sigma$  muestra la tendencia esperada, ya que ambos errores crecen al aumentar el ruido. El parámetro  $\gamma$  resulta más difícil de inferir y su error crece más rápido, ya que el decaimiento se confunde con las fluctuaciones aleatorias a medida que la señal pierde amplitud.

Cuando el modelo se ajusta tan finamente a los datos de entrenamiento que termina aprendiendo también su ruido en lugar de la relación física subyacente. Se manifiesta como una brecha entre el error de entrenamiento, que es pequeño, y el error de validación, que es grande.

## 2. Problema 2: Terapia con protones

Se construye un simulador Monte Carlo del transporte de protones en un fantoma de agua para caracterizar el pico de Bragg. Definimos las constantes físicas en MeV y cm, y las propiedades del agua se fijan con  $Z = 10$ ,  $A = 18,015$  g/mol,  $I = 75$  eV y  $\rho = 1$  g/cm<sup>3</sup>.

#### 2.1. a) Conceptos teóricos:

**Pico de Bragg:** La curva de dosis  $D(z)$  de un protón presenta un máximo muy pronunciado justo al final de su recorrido. Esto ocurre debido a la dependencia  $-dE/dx \propto 1/\beta^2$ , ya que a medida que el protón se frena, su velocidad disminuye y la pérdida de energía por unidad de longitud crece. Casi al detenerse deposita la mayor parte de su energía en una región estrecha, formando el pico. Este comportamiento es lo opuesto al de los fotones y es la base de la terapia con protones, porque permite concentrar la dosis en el tumor protegiendo el tejido sano que está más adelante.

**Ecuación de Bethe–Bloch:** Esta ecuación describe la pérdida media de energía por unidad de longitud, que está dada por:

$$-\frac{dE}{dx} = Kz^2 \frac{Z}{A} \frac{1}{\beta^2} \left[ \frac{1}{2} \ln \frac{2m_e c^2 \beta^2 \gamma^2 T_{\text{máx}}}{I^2} - \beta^2 \right]. \quad (5)$$

Aquí  $z$  es la carga de la partícula incidente,  $Z/A$  caracteriza la densidad electrónica del medio,  $\beta = v/c$  y  $\gamma$  son los factores relativistas,  $m_e c^2$  la energía en reposo del electrón,  $T_{\text{máx}}$  la energía máxima transferible a un electrón en un solo choque, e  $I$  el potencial medio de excitación  $I \approx 75$  eV (para el agua), que se encarga de parametrizar la energía promedio transferida por colisión.

El factor  $z^2$  aparece porque la fuerza de Coulomb sobre los electrones del medio es proporcional a la carga de la partícula, y la energía transferida va con el cuadrado de esa fuerza. El factor  $1/\beta^2$  refleja que una partícula lenta pasa más tiempo cerca de cada electrón, transfiriéndole más energía, es este término el que produce el aumento de la pérdida de energía al final del recorrido y, con ello, el pico de Bragg.

**Energy straggling:** La pérdida de energía fluctúa estadísticamente, porque el número y la magnitud de las colisiones varían entre protones. En la aproximación gaussiana de Bohr la varianza de esa pérdida por paso esta dada por:

$$\sigma_E^2 = 4\pi r_e^2 (m_e c^2)^2 N_e \frac{z^2}{\beta^2} \Delta x, \quad (6)$$

donde  $N_e$  es la densidad electrónica del medio. Este efecto ensancha el pico de Bragg y desvanece su posición exacta.

**Dispersión Coulombiana múltiple:** Las colisiones con los núcleos desvían angularmente la trayectoria del protón. En la aproximación de Highland, la desviación angular acumulada esta dada por:

$$\theta_0 = \frac{13,6 \text{ MeV}}{\beta c p} z \sqrt{\frac{\Delta x}{X_0}} [1 + 0,038 \ln(\Delta x/X_0)], \quad (7)$$

donde  $X_0$  la longitud de radiación del medio. Este efecto ensancha lateralmente el haz.

## 2.2. b) Implementación de Bethe–Bloch y rango CSDA:

La función `funcion_bethebloch` implementa la ecuación anterior. A partir de la energía cinética  $T$  calcula  $\gamma = 1 + T/m_p$ , luego  $\beta^2 = 1 - 1/\gamma^2$  y la energía máxima transferida, que esta dada por:

$$T_{\text{máx}} = \frac{2m_e \beta^2 \gamma^2}{1 + 2\gamma m_e/m_p + (m_e/m_p)^2}, \quad (8)$$

Finalmente evaluamos el corchete logarítmico. Se usa  $z = 1$  porque la partícula es un protón. El primer gráfico muestra  $-dE/dx$  en escala log–log, donde se aprecia el mínimo de ionización y el fuerte crecimiento a bajas energías.

El rango CSDA se obtiene integrando el inverso del poder de frenado, de modo que:

$$R_{\text{CSDA}}(E_0) = \int_0^{E_0} \frac{dE}{(-dE/dx)(E)}. \quad (9)$$

La función `funcion_CSDA` genera una grilla de energías entre 0.05 MeV y  $E_0$ , queremos evitar el cero porque ahí la integral diverge, evalúa Bethe-Bloch sobre esa grilla y aplica la regla del trapecio con `np.trapezoid` sobre  $1/(-dE/dx)$ . El límite inferior en 0.05 MeV introduce un pequeño error sistemático, pero a cambio nos mantiene la integral finita.

La comparación con la base PSTAR del NIST se resume en la Tabla 1.

Cuadro 1: Rango CSDA calculado frente a los valores de referencia NIST PSTAR.

| $E_0$ [MeV] | $R_{\text{CSDA}}$ calculado [cm] | NIST [cm] | Error relativo |
|-------------|----------------------------------|-----------|----------------|
| 50          | $\approx 2,2$                    | 2.207     | $\sim 1 \%$    |
| 150         | $\approx 15,8$                   | 15.76     | $\sim 1 \%$    |
| 250         | $\approx 38$                     | 38.00     | $\sim 1 \%$    |

El segundo gráfico muestra el rango CSDA como función de la energía inicial (en mm) junto con los tres puntos de referencia del NIST, que caen sobre la curva calculada.

### 2.3. c) Simulación Monte Carlo sin straggling:

Para acelerar la simulación, el script pre-calcula tablas de  $-dE/dx$  y de  $\beta^2$  sobre 300.000 valores de energía y construye interpoladores lineales (`interp1d`). Así, durante la simulación no se reevalúa Bethe–Bloch en cada paso, sino que se consulta la tabla.

La función `simular` es el motor del transporte. Divide la profundidad máxima en cortes de espesor  $\Delta x = 0,1$  mm, inicializa  $N = 10^4$  protones con la misma energía  $E_0 = 150$  MeV y los avanza corte a corte. En cada corte, para los protones todavía activos, calcula la pérdida media de energía como:

$$\langle \Delta E \rangle = \left( -\frac{dE}{dx} \right) \Delta x, \quad (10)$$

deposita esa energía en el bin correspondiente, la resta de la energía de cada protón y desactiva los que bajan de un umbral de 0.05 MeV. El proceso se detiene cuando ya no quedan protones con energía. En este modo sin straggling todos los protones pierden lo mismo en cada paso, por lo que se frenan a la misma profundidad.

La salida del simulador es la dosis depositada en función de la profundidad. Al buscar el máximo (`np.argmax`) se identifica la posición del pico de Bragg, que se compara con el  $R_{\text{CSDA}}$  obtenido en el inciso anterior. Ambos valores coinciden de forma muy cercana para  $E_0 = 150$  MeV, lo cual es la verificación esperada, ya que el pico de Bragg debe aparecer justo donde el protón agota su energía, que es lo que predice el rango CSDA. El tercer gráfico muestra la dosis normalizada con líneas verticales marcando el pico simulado y el rango teórico.

### 2.4. Simulación con straggling (inciso d)

La misma función `simular` se ejecuta ahora con `straggling=True`. En este modo, además de la pérdida media, en cada paso se calcula la desviación estándar a partir de la fórmula de Bohr:

$$\sigma_E^2 = 4\pi r_e^2 (m_e c^2)^2 N_e \frac{1}{\beta^2} \Delta x, \quad (11)$$

A esto se le suma a la pérdida media un ruido gaussiano  $\mathcal{N}(0, \sigma_E^2)$ . La energía perdida en cada paso pasa a ser una variable aleatoria  $\Delta E \sim \mathcal{N}(\langle \Delta E \rangle, \sigma_E^2)$ . La energía depositada se recorta para que nunca exceda la energía disponible del protón (`np.clip`). Como ahora cada protón pierde una cantidad ligeramente distinta, no todos se detienen exactamente en la misma profundidad, y el pico se ensancha.

Para calcular el ensanchamiento, la función `ensanchamiento` mide el ancho a media altura (FWHM) del pico, parte del máximo y camina hacia ambos lados hasta que la dosis cae por debajo de la mitad del valor pico. Suponiendo un perfil gaussiano, se convierte el FWHM a desviación estándar usando:

$$\sigma_R = \frac{\text{FWHM}}{2,355}. \quad (12)$$

Al comparar ambos casos se observa que el pico con straggling es más ancho y algo más bajo que el ideal, mientras que su posición media se mantiene igual. El cuarto gráfico superpone ambos perfiles.

**Relevancia clínica:** El ensanchamiento  $\sigma_R$  producido por el straggling significa que el final del recorrido de los protones no es un punto exacto sino una región difusa. Clínicamente esto importa ya que el borde distal del pico de Bragg define hasta dónde llega la dosis, si ese borde es difuso, parte de la dosis puede caer fuera del tumor, sobre tejido sano que está justo detrás. Por eso, al planificar un tratamiento, este ensanchamiento debe tenerse en cuenta junto con los márgenes de seguridad para garantizar que el tumor reciba la dosis prescrita sin dañar estructuras críticas vecinas.